SDE-2 Take-Home Assignment: Unified Conversations Inbox

Overview

Businesses talk to their customers across SMS, Email and WhatsApp. Agents want **one inbox** where every message from a contact, across every channel, appears in a single conversation thread, in real time.

Your task is to build the **backend service** for this inbox.

Submission deadline	Within 72 hours of receiving this assignment
Expected effort	5–6 hours
Preferred stack	Node.js + TypeScript (any database: MongoDB / Postgres; Redis optional)
AI tools	Allowed (Cursor, Claude, Copilot, etc.); see the AI Usage section

Channel providers (Twilio, Email, WhatsApp) should be **mocked**. You do not need real accounts. Treat their callbacks as simple webhook payloads, as shown below.

Requirements

1. Receive Inbound Messages

`POST /webhooks/inbound`

```JSON
{
  "providerMessageId": "SM_abc123",
  "locationId": "loc_1",
  "channel": "sms",
  "from": "+91 98765 43210",
  "to": "+91 80000 00000",
  "body": "Is the offer still valid?",
  "sentAt": "2026-10-28T10:15:00Z"
}
```

- `channel` can be `sms`, `whatsapp` or `email`.
- Find or create the **contact**, matched by normalized phone number (SMS / WhatsApp) or email (Email), scoped to the `locationId`.
- Find or create **one conversation per contact per location**. Messages from all channels go into the same conversation.
- Providers may **retry webhooks**. The same `providerMessageId` must never be stored twice.
- Update the conversation's `lastMessageAt`, `lastMessagePreview` and `unreadCount`.

2. Send Outbound Messages

`POST /conversations/:id/messages`

```JSON
{ "channel": "whatsapp", "body": "Yes, valid till Friday!" }
```

- Save the message with status `queued` and send it **asynchronously** through a mock provider.
- The mock provider should **fail randomly about 20% of the time**.
- Retry failed sends with backoff, up to 3 attempts, then mark the message `failed`.

3. Handle Delivery Status Callbacks

`POST /webhooks/status`

```
JSON
{ "providerMessageId": "WA_xyz", "status": "delivered", "timestamp":
"2026-10-28T10:16:00Z" }
```

- Status lifecycle: `queued → sent → delivered → read`. A message can become `failed` at any point.
- Callbacks may arrive **out of order** (for example, `delivered` before `sent`). A message's status must **never move backwards**.
- A callback may arrive for a message your system has not finished saving yet. Handle this safely.

4. Inbox APIs

Method	Endpoint	Description
GET	`/locations/:locationId/conversations`	List conversations sorted by latest message. Supports filters `status` (open/closed) and `assignedTo`. Use **cursor-based pagination**.
GET	`/conversations/:id/messages`	Messages in chronological order, with cursor-based pagination.
POST	`/conversations/:id/read`	Mark the conversation as read (reset unread count).
PATCH	`/conversations/:id`	Assign to an agent, or open/close the conversation.

- A new inbound message on a **closed** conversation should reopen it.
- Pagination must not return duplicates or skip items when new messages arrive between page loads.

5. Real-Time Updates

- Push new messages and status changes to connected agents using **WebSocket or SSE**.
- Agents must only receive events for **their own location**.

6. Tests

Include automated tests that cover at least:

- A duplicate inbound webhook is stored only once.
- Out-of-order status callbacks result in the correct final status.
- 50 inbound messages sent **in parallel** to one conversation result in `unreadCount = 50`.
- Pagination remains correct when new messages are inserted between page requests.

Bonus (Optional)

- Message search: `GET /locations/:locationId/messages/search?q=offer`
- Typing indicators or agent presence over the real-time channel.
- A short design note: how would your system handle **10,000 inbound messages per second**, and how would real-time push work across **multiple server instances**?

Submission

Share a **GitHub repository link** (public, or private with access granted) containing:

1. **Source code and tests**
2. **README.md** covering:
    - How to run the service and the tests
    - Your data model and indexes
    - How you handled idempotency, out-of-order statuses, unread counts and real-time updates
    - Trade-offs you made and what you would improve with more time
3. **AI_USAGE.md** covering:
    - Which AI tools you used and for what
    - At least one specific example where AI-generated code was wrong or needed changes, and how you identified and fixed it

---

What Happens Next

After submission, we will review your design and code. Please make sure you understand every part of your submission, including any AI-generated code.

If anything in this assignment is unclear, make a reasonable assumption and note it in your README.

Good luck! We look forward to seeing your solution.